

WEEK-04

Implement Knapsack problem using dynamic programming.

```
#include <stdio.h>
```

```
struct Item {  
    int weight;  
    int profit;  
    float ratio;  
};
```

```
void sortItems(struct Item items[], int n) {  
    int i, j;  
    struct Item temp;  
    for (i = 0; i < n - 1; i++) {  
        for (j = i + 1; j < n; j++) {  
            if (items[i].ratio < items[j].ratio) {  
                temp = items[i];  
                items[i] = items[j];  
                items[j] = temp;  
            }  
        }  
    }  
}
```

```
float fractionalKnapsack(int capacity, struct Item items[], int n) {  
    float totalProfit = 0.0;  
    int i;  
    for (i = 0; i < n; i++) {  
        if (capacity >= items[i].weight) {  
            capacity -= items[i].weight;  
            totalProfit += items[i].profit;  
        } else {  
            totalProfit += items[i].ratio * capacity;  
            break;  
        }  
    }  
    return totalProfit;  
}
```

```
int main() {  
    int n, i, capacity;  
    printf("Enter the number of items: ");  
    scanf("%d", &n);
```

```

struct Item items[n];

for (i = 0; i < n; i++) {
    printf("Enter profit and weight of item %d: ", i + 1);
    scanf("%d %d", &items[i].profit, &items[i].weight);
    items[i].ratio = (float)items[i].profit / items[i].weight;
}

printf("Enter the capacity of the knapsack: ");
scanf("%d", &capacity);

sortItems(items, n);

float maxProfit = fractionalKnapsack(capacity, items, n);
printf("Maximum profit in the knapsack = %.2f\n", maxProfit);

return 0;
}

```

OUTPUT:

```

● PS C:\Users\SHALINI K A\Desktop\ADA> gcc Knapsack.c -o Knapsack
● PS C:\Users\SHALINI K A\Desktop\ADA> ./Knapsack
Enter the number of items: 4
Enter profit and weight of item 1: 20
10
Enter profit and weight of item 2: 30
40
Enter profit and weight of item 3: 20
60
Enter profit and weight of item 4: 40
30
Enter the capacity of the knapsack: 50
Maximum profit in the knapsack = 67.50

```

Leet Code: Maximum Units on a Truck.

```
class Solution {
public:
    int maximumUnits(vector<vector<int>>& boxTypes, int truckSize) {
        sort(boxTypes.begin(), boxTypes.end(), [](vector<int>& a, vector<int>& b){
            return a[1] > b[1];
        });

        int totalUnits = 0;

        for (auto &box : boxTypes) {
            int boxesToTake = min(box[0], truckSize);
            totalUnits += boxesToTake * box[1];
            truckSize -= boxesToTake;
            if (truckSize == 0) break;
        }

        return totalUnits;
    }
};
```